

Cross-Org Review System — Design v2 (source of truth)

Status: locked 2026-06-12, execution in phases (P0 → P3). Supersedes the v1 draft of 2026-05-29; the v1 settled-decisions table is kept below as lineage. The major v2 changes: the enforcement model is rebuilt on the **post-RLS-flip reality** (user-context RLS is the primary boundary, not a service-role resolver), promotion is a **trimmed copy** rather than live grant projection, and the phasing is re-cut so cross-org value ships before the heavy definition machinery.

ArtHound reviews graduate from a flat per-org note into the **method and record of sign-off and delivery** for an asset. A review is ArtHound-native (never synced), always tied to at least one canonical asset, and serves three routines:

1. **Internal** daily craft/team review (single org, never shared).
 2. **Cross-org ritual** — ad-hoc vendor→studio submissions and the link's required protocol.
 3. **Formal delivery** — a cross-org review terminating in the studio accepting delivery.
-

1. Requirements (2026-06-12)

1. Studios determine **required submissions** at the vendor/handshake (link) level. Eventually vendors must explicitly accept these; for now they are visible and actionable for vendors.
 2. Vendors can **always** generate ad-hoc review requests for the studio (on an active link).
 3. Vendors have **internal reviews** that can be **promoted** into cross-org reviews. Comments and vendor-specific data must be trimmable at promotion — same mental model as payload templates, but for review fields and visibility. Granularity/flexibility of this system is expected to grow significantly.
 4. **Security first** — respect the live RLS implementation (four runtime identities, user-context under RLS, D-pattern for cross-org tables, DEFINER RPCs for partner actions).
 5. Every review ties to **at least one canonical asset**; each side's asset context is resolved from their own view (studio: `replicated_assets` ; vendor: dispatch payload).
 6. The system includes **comments**, with cross-org sharing and sanitization in mind.
 7. **Status stays simple for now** (current vocabulary: Pending / In Review / Approved / Changes Requested); eventually studio/vendor-definable.
-

2. Locked decisions

#	Decision	Choice
1	Review unit	Many reviews per asset — the asset accrues a review history; current sign-off = latest accepted delivery review. (<i>carried from v1</i>)
2	Promotion model	Hybrid promoted copy — promoting an internal review creates a NEW cross-org review row (<code>promoted_from_review_id</code>); a trim template selects which fields/comments/attachments copy over. The internal review stays private forever. Schema stays grant-compatible so per-record grants can layer on later.
3	Required submissions	One protocol per link — the link references one studio-owned <code>review_workflow_def</code> ; its ordered <code>review_step_def</code> rows ARE the required submissions per asset. A vendor fulfils one by submitting/promoting a cross-org review tagged with the <code>step_def_id</code> .
4	Comments	Lanes + promote — <code>visibility ∈ internal\ shared</code> . Internal reviews allow only <code>internal</code> . On cross-org reviews both orgs get both lanes. A comment can be flipped internal→shared later (one-way, event-logged). Trim template picks internal comments to copy at promotion.
5	Asset linkage	<code>review_assets</code> junction from day one (m2m). <code>asset_reviews.canonical_asset_id</code> stays NOT NULL as the primary anchor and is mirrored into the junction so reads join only the junction.
6	Statuses	Free-text <code>status</code> + the existing frontend <code>STATUS_OPTS</code> . <code>review_status_def</code> (org-definable vocab + meta-status mapping) is deferred ; nothing in v2 blocks adding it.
7	Authoring UI	Simple ordered list protocol editor (name + description + order) with a seeded default ("Delivery Review"). Stages, actor tagging, branching, per-step statuses deferred.
8	Acceptance	Final phase (P3) — acceptance RPC + frozen snapshot + revision chain. Until then <code>Approved</code> is the informal sign-off.
9	Enforcement	User-context RLS is the primary boundary (post-flip). Partner-side actions (e.g. studio transitions a vendor submission's status) go through <code>SECURITY DEFINER</code> RPCs owned by <code>arthound_rpc</code> — the established D-pattern. The v1 "service-role route resolver" section is dead.
10	Re-delivery	New linked review (<code>revision_of_review_id</code>); history immutable. (<i>carried from v1</i>)
11	Acceptance freeze	Snapshot of review + delivered asset data/attachment refs (content-addressed <code>sha256</code> , never recopied). (<i>carried from v1</i>)

Behavior defaults (confirmed 2026-06-12)

1. **Link cancellation:** cross-org reviews stay readable to both parties after the link is cancelled (delivered records must not vanish); new cross-org writes on that link are blocked. Matches the `link_cancellation_audit` philosophy, not payload revoke.
2. **Deletion:** the creator may delete their own reviews in P0–P2. From P3, accepted reviews are immutable (no update/delete).

3. **Symmetry:** RLS permits studio-authored cross-org reviews too (cheap, future-proof), but v1 UI only builds the vendor-side create/promote flows.
 4. **Requirement applicability:** the link protocol checklist applies to assets with a live (non-revoked) dispatch on that link.
-

3. What exists today (verified 2026-06-12)

- `asset_reviews` is flat: `{studio_id, canonical_asset_id, author_org_type, author_org_id, title, description, status (free text), created_by_email, created_by_user_id, shared_at (unused stub), created_at, updated_at}`. Route layer (`routes/reviews.py`) walls by author org; `_enrich` resolves asset context per side (studio: `replicated_assets` ; vendor: `dispatch payload_data`) — that dual-view pattern is kept and extended.
 - **Live RLS gap:** policy `ar_sel` (migration `20260530000003`) lets a studio SELECT **all** reviews on its assets including vendor-authored ones. Routes filter them out, but at the RLS layer vendor reviews on dispatched assets are visible to the studio — the opposite of requirement 3. **P0 replaces this policy.**
 - `review_attachments` → Supabase Storage, inline read-only streaming. Reused as-is; P0 folds in the `uploaded_by_user_id` hardening TODO (delete currently keys on email).
 - `studio_vendor_links.review_collaboration_mode` $\in$ `{none, isolated, collaborative}` exists but becomes **vestigial**: requirement 2 makes ad-hoc cross-org reviews always available on an active link. Leave the column, stop reading it. The link's protocol FK (P2) is the real cross-org review config.
 - Reusable patterns: `payload_templates` (sender-side field allowlist → frozen artifact), org-scope stack (dual FK + generated `owner_key` , e.g. `workflow_steps`), D-pattern (dual-party SELECT / owner-only writes / cross-org transitions via `arthound_rpc` RPCs), `link_cancellation_audit` (append-only, no INSERT policy).
-

4. Entity model

`asset_reviews` (extended — one container for internal AND cross-org)

Column	Phase	Notes
<code>scope</code> text NOT NULL DEFAULT 'internal'	P0	CHECK (scope IN ('internal', 'cross_org'))
<code>link_id</code> uuid REFERENCES <code>studio_vendor_links</code>	P0	NOT NULL when <code>scope='cross_org'</code> (CHECK)
<code>promoted_from_review_id</code> uuid REFERENCES <code>asset_reviews</code>	P0	set when created via promotion
<code>step_def_id</code> uuid REFERENCES <code>review_step_def</code>	P2	which required submission this fulfils; NULL = ad-hoc
<code>revision_of_review_id</code> uuid REFERENCES <code>asset_reviews</code>	P3	re-delivery chain
<code>accepted_at</code> timestampz, <code>accepted_by_user_id</code> uuid, <code>frozen_snapshot</code> jsonb	P3	acceptance freeze
<code>shared_at</code>	P0	dropped (unused stub, superseded)

New tables

- `review_assets` (P0) — {`review_id`, `canonical_asset_id`, `created_at`}, PK (`review_id`, `canonical_asset_id`). Holds **every** linked asset including the primary (backfilled from `canonical_asset_id`); reads join only the junction. App layer keeps the primary mirrored.
- `review_comments` (P0) — {`id`, `review_id`, `author_org_type`, `author_org_id`, `author_user_id`, `author_email`, `body`, `visibility` text NOT NULL DEFAULT 'internal' CHECK (visibility IN ('internal', 'shared')), `shared_at`, `copied_from_comment_id` uuid, `edited_at`, `created_at`}. Lane flip is one-way internal→shared, evented.
- `review_events` (P0) — append-only audit: {`id`, `review_id`, `subject_type`, `subject_id`, `event_type`, `actor_user_id`, `actor_org_type`, `actor_org_id`, `detail` jsonb, `created_at`}. Event types: `created`, `updated`, `status_changed`, `deleted`, `comment_added`, `comment_shared`, `promoted`, `attachment_added`, `requirement_tagged`, `accepted`, `revision_created`. **No INSERT policy** — written via `arthound_system` / DEFINER RPCs only.
- `review_trim_templates` (P1) — vendor-owned, mirrors `payload_templates`: {`id`, `vendor_id`, `link_id` uuid NULL (per-link default), `name`, `config` jsonb, `created_at`, `updated_at`}. `config` is deliberately jsonb — {`fields`: { ... }, `comments`: 'none'|'shared'|'selected', `attachments`: 'none'|'all'|'selected'} today; granularity will grow.
- `review_workflow_def` (P2) — org-scope stack convention (dual FK `studio_id` / `vendor_id` generated `owner_key`) even though v1 authoring is studio-only: {`id`, `studio_id`, `vendor_id`, `owner_key`, `name`, `description`, `is_default`, `archived_at`, `created_at`, `updated_at`}.

- `review_step_def` (P2) — {id, workflow_def_id, name, description, sort, archived_at, created_at}. Deferred columns slot in later: `stage`, `actor`, `is_shared_gate`, `default_status_def_id`.

Link extension (P2)

- `studio_vendor_links.review_protocol_def_id` uuid REFERENCES `review_workflow_def` — the sanctioned protocol.
- `studio_vendor_links.protocol_acknowledged_at` timestampz, `protocol_acknowledged_by` uuid — stub for the future explicit vendor-acceptance flow (v1: visible + actionable only).

`review_attachments` (P0 hardening)

- Add `uploaded_by_user_id` uuid (backfill best-effort from email), key DELETE on it.
- Add `copied_from_attachment_id` uuid (P1, promotion provenance). Promoted attachments are **new rows referencing the same content-addressed storage path** (no byte copy); blob deletion must first check no other row references the path.

5. Security model

Default-private. User-context RLS is the enforced boundary; routes run as the caller.

- `asset_reviews` **SELECT** (replaces over-broad `ar_sel`): `is_my_org(author_org_type, author_org_id)` OR (`scope='cross_org'` AND caller's org is a party to `link_id` — any link status, per behavior default 1). This **removes** the studio's blanket RLS visibility of vendor-authored reviews on its assets. (No user-visible behavior change — routes already filtered them.)
- **INSERT:** own org as author; for `scope='cross_org'` additionally a party to an **active** link, and the link must connect the asset's studio to the authoring/receiving orgs.
- **UPDATE/DELETE:** owner org only (delete additionally creator-keyed). Partner actions — studio transitioning a vendor submission's status — via DEFINER RPC (`review_set_status`) validating link party + allowed transition, writing the event in the same transaction.
- `review_comments` : SELECT = author org OR (`visibility='shared'` AND parent review visible cross-org to caller's org) — EXISTS chain to `asset_reviews` + link. INSERT requires the parent review to be visible and lane rules to hold (internal reviews: `internal` only). Lane flip internal→shared via RPC or trigger-guarded UPDATE, one-way, evented. Edit/delete: author user.
- `review_assets` / `review_events` : visibility mirrors the parent review (EXISTS). `review_events` has no INSERT/UPDATE/DELETE policies (system/RPC writes only).
- `review_workflow_def` / `review_step_def` : own-org read/write via `owner_key` ; PLUS partner SELECT when the def is referenced by `review_protocol_def_id` of a link the caller's org is party to (vendors must see the studio's protocol).
- `review_trim_templates` : vendor-own only.
- **Promotion** is one ACID DEFINER RPC `promote_review(internal_review_id, link_id, step_def_id?, trim jsonb)` : validate caller ∈ vendor org owning the internal review; link active; all

junction assets belong to the link's studio → insert trimmed cross-org review → copy selected comments (`copied_from_comment_id` , `visibility='shared'`) → reference selected attachments → events on both reviews → return new id.

- **Acceptance (P3)** is one ACID DEFINER RPC: validate required protocol steps fulfilled → assemble `frozen_snapshot` (review state + delivered asset data + attachment `sha256` refs) → set `accepted_at/by` → event. Accepted reviews immutable (policy-enforced).
 - **Ops discipline:** FORCE RLS on all new tables; grants + policies registered in `scripts/rls_grant_audit.py` ; persona-matrix tests extended (partner-without-link → empty, internal lane → empty cross-org, cancelled link → read-only). `arthound_system` gets SELECT on new tables up front (future notification loop) — avoiding RLS regression class #6/#7.
-

6. Flows

1. **Internal review** (routine 1): `scope='internal'` , comments in the internal lane, zero cross-org surface. Cheapest path, unchanged UX plus threads.
2. **Ad-hoc cross-org request** (req 2): vendor creates a review with `scope='cross_org'` + `link_id` directly (no internal precursor), or promotes an internal one. Studio sees it in their cross-org inbox, comments in the shared lane, sets status via the transition RPC. The review itself is the request artifact — no separate request object.
3. **Required submissions** (req 1): studio edits the link's protocol (ordered named steps). Per dispatched asset, both sides see a computed checklist: each step def × asset → the latest cross-org review tagged with that `step_def_id` (none = unfulfilled; status `Approved` = complete). Computed at read time, no materialized state.
4. **Formal delivery** (routine 3, P3): the protocol's final step's review is accepted by the studio via the acceptance RPC → snapshot freeze. Rejection/rework = new review with `revision_of_review_id` ; the prior stays frozen.

Asset context per side (req 5): keep/extend `_enrich` — studio resolves junction assets from `replicated_assets` ; vendor resolves from received dispatch `payload_data` . The review shows each side their own view of the canonical asset(s) plus what was sent.

7. Phasing (each independently shippable)

Phase	Scope	Delivers
P0 — Foundations + comments	<code>scope / link_id / promoted_from</code> columns; drop <code>shared_at</code> ; <code>review_assets</code> + backfill; <code>review_comments</code> ; <code>review_events</code> ; RLS replacement (fixes <code>ar_sel</code> over-breadth); <code>uploaded_by_user_id</code> hardening; comments CRUD + thread UI (internal)	Internal reviews gain threads; live RLS gap closed. No cross-org behavior change.
P1 — Cross-org + promotion (reqs 2, 3, 6)	Dual-party RLS for <code>scope='cross_org'</code> ; ad-hoc vendor→studio creation; <code>promote_review</code> RPC; <code>review_trim_templates</code> + promote modal (payload-template UX); studio inbox; <code>review_set_status</code> RPC; shared comment lane	Vendors submit/promote; studios receive, comment, set status.
P2 — Link protocol (req 1)	<code>review_workflow_def / review_step_def</code> ; link <code>review_protocol_def_id</code> + acknowledgment stub; studio ordered-list editor in link UI; computed per-asset checklist; <code>step_def_id</code> tagging	Studios define required submissions; vendors see and fulfil them.
P3 — Formal acceptance	Acceptance RPC + <code>frozen_snapshot</code> ; immutability of accepted reviews; <code>revision_of_review_id</code> chain; asset review-history surface	Routine 3: immutable delivery record; revisions as new reviews.

8. Deferred (documented, not built)

- **Per-record grants** (`review_grant`) — the v1 model; the promoted-copy schema is grant-compatible (stable per-record ids on reviews/comments/attachments; visibility checks centralized in policies that can grow an `OR EXISTS (grant)` arm).
- **Org-definable statuses** (`review_status_def` + meta-status mapping) — req 7's future.
- **Explicit vendor acceptance of the link protocol** — stub columns ship in P2.
- **Vendor-owned protocols**, stages/actors/branching in step defs.
- **Studio↔studio reviews** — org-typed columns generalize; handshake is studio↔vendor today.
- **Notifications** — hook into the notification-system TODO; never hardcode delivery here.
- **Cadence/scheduling** of rituals; field-level custom review data; real-time collaboration.

9. Risks

- **Cross-org leak via policy bug** — dominant risk. Mitigation: small policy surface (lane + link-party checks, no grant subqueries in v2), persona-matrix tests for every new table, `rls_grant_audit` keeps file-vs-DB drift visible (regression class #8).

- **EXISTS-chain query cost** — index `review_comments(review_id)`, `asset_reviews(link_id)`, `asset_reviews(step_def_id)`, `review_assets(canonical_asset_id)`.
- **Shared storage paths** — promoted attachment rows reference the internal blob's path; deletion must check for other referencing rows or the shared copy 404s.
- **Promotion divergence** — the internal review keeps evolving after promotion by design; provenance (`promoted_from_review_id`, `copied_from_comment_id`) is the audit trail, and re-promotion (P3 revision chain) is the refresh mechanism.
- **Checklist read cost** — computed per (link, dispatched assets × step defs); fine at current scale, materialize later if needed.

10. Lineage — v1 settled decisions (2026-05-29)

Kept for the record; items struck are superseded by v2.

#	v1 decision	v2 status
1	Many reviews per asset	Kept (decision 1)
2	Private lanes + shared gates	Kept in spirit — lanes are comment-level; gates become protocol step defs
3	Explicit per record grants everywhere	Superseded by promoted-copy + lanes; grants deferred
4	Route resolver (service role) + RLS depth	Superseded — user-context RLS primary + DEFINER RPCs (post-flip)
5	Linear/staged steps	Kept, simplified — ordered list in v1; stages deferred
6	Re-delivery = new linked review	Kept (decision 10)
7	Acceptance freeze = snapshot	Kept (decision 11)

11. ArtHound-principle check

- **Security/integrity first:** default-private, user-context RLS as the enforced boundary, ACID RPCs for every cross-org mutation, append-only events, fixes a live RLS gap in P0.
- **Everything links to its canonical asset:** `canonical_asset_id` NOT NULL + junction.
- **Named fields = universal truths only:** statuses stay data; protocol steps are def rows.
- **No cross-org flow without explicit authorization:** authorship on an active link, promotion RPC, or protocol fulfilment — every crossing is an event row.
- **Prescriptive baseline, modular where it counts:** seeded default protocol; trim templates per link; deferred machinery has reserved slots, not rework.